MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	Sanjeev Khatri		
Subject Name with Code	4AD4-06 Introduction to Java Programming		
Semester	4th	**Session**	2022-23 (Even)
Branch	AIDS	**Batch**	A
Unit No.	5	**Submission Date**	27-Mar-23

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	I/O Basics, Reading Console Input, Writing Console Output, Reading and Writing Files	1 - 9	9
2	Applets: Applet Fundamentals, Applet Architecture	10 - 10	1
3	The HTML Applet tag, Passing parameters to Applets	11 - 13	3
4	Networking basics, Java and the Net	14 - 15	2
5	TCP/IP Client Sockets URL, URL Connection, TCP/IP Server Sockets	16 - 21	6
6	Database connectivity	22 -27	6

Faculty

HOD

27/03/2023

Dean Academics

UNIT-5 (I/O Operations)

LECTURE - 1

I/O Basics, Reading Console Input, Writing Console Output, Reading and Writing Files

Java's I/O system is quite large, containing many classes, interfaces, and methods. Java programs perform I/O through streams. An I/O stream is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system. All streams behave in the same manner, even if the actual physical devices they are linked to differ.

Byte Streams and Character Streams

Byte streams provide a convenient means for handling input and output of bytes. They are used when reading or writing binary data. They are especially helpful when working with files. Character streams are designed for handling the input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.

The Byte Stream Classes

Byte streams are defined by using two class hierarchies. At the top of these are two abstract classes: **InputStream** and **OutputStream**. **InputStream** defines the characteristics common to byte input streams and **OutputStream** describes the behavior of byte output streams.

Byte Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements **InputStream**
FilterOutputStream	Implements **OutputStream**
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains **print( )** and **println( )**
PushbackInputStream	Input stream that allows bytes to be returned to the stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

Byte Stream Classes in java.io.

The Character Stream Classes

Character streams are defined by using two class hierarchies topped by these two abstract classes: Reader and Writer. Reader is used for input, and Writer is used for output. Concrete classes derived from Reader and Writer operate on Unicode character streams.

Character Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains **print( )** and **println( )**
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

The Character Stream I/O Classes in java.io.

The Predefined Streams

All Java programs automatically import the java.lang package. This package defines a class called System, which encapsulates several aspects of the runtime environment. It also contains three predefined stream variables called **in**, **out**, and **err**. **System.out** refers to the standard output stream. **System.in** refers to standard input, which is by default the keyboard. **System.err** refers to the standard error stream. **System.in** is an object of type InputStream, **System.out** and **System.err** are objects of type **PrintStream**.

Using the Byte Stream

At the top of the byte stream hierarchy are the **InputStream** and **OutputStream** classes. In general, the methods in InputStream and OutputStream can throw an **IOException** on error.

Method	Description
int available( )	Returns the number of bytes of input currently available for reading.
void close( )	Closes the input source. Subsequent read attempts will generate an **IOException**.
void mark(int *numBytes*)	Places a mark at the current point in the input stream that will remain valid until *numBytes* bytes are read.
boolean markSupported( )	Returns true if **mark( )**/**reset( )** are supported by the invoking stream.
static InputStream nullInputStream( )	Returns an open, but null stream, which is a stream that contains no data. Thus, the stream is always at the end of the stream and no input can be obtained. The stream can, however, be closed. (Added by JDK 11.)
int read( )	Returns an integer representation of the next available byte of input. −1 is returned when an attempt is made to read at the end of the stream.
int read(byte *buffer*[ ])	Attempts to read up to *buffer.length* bytes into *buffer* and returns the actual number of bytes that were successfully read. −1 is returned when an attempt is made to read at the end of the stream.
int read(byte *buffer*[ ], int *offset*, int *numBytes*)	Attempts to read up to *numBytes* bytes into *buffer* starting at *buffer*[*offset*], returning the number of bytes successfully read. −1 is returned when an attempt is made to read at the end of the stream.
byte[ ] readAllBytes( )	Reads and returns, in the form of an array of bytes, all bytes available in the stream. An attempt to read at the end of the stream results in an empty array.
byte[ ] readNBytes(int *numBytes*)	Attempts to read *numBytes* bytes, returning the result in a byte array. If the end of the stream is reached before *numBytes* bytes have been read, then the returned array will contain less than *numBytes* bytes. (Added by JDK 11.)
int readNBytes(byte *buffer*[ ], int *offset*, int *numBytes*)	Attempts to read up to *numBytes* bytes into *buffer* starting at *buffer*[*offset*], returning the number of bytes successfully read. An attempt to read at the end of the stream results in zero bytes being read.
void reset( )	Resets the input pointer to the previously set mark.
long skip(long *numBytes*)	Ignores (that is, skips) *numBytes* bytes of input, returning the number of bytes actually ignored.
long transferTo(OutputStream *outStrm*)	Copies the contents of the invoking stream to *outStrm*, returning the number of bytes copied.

List of a few Methods defined by **InputStream** class.

Method	Description
void close( )	Closes the output stream. Subsequent write attempts will generate an **IOException**.
void flush( )	Causes any output that has been buffered to be sent to its destination. That is, it flushes the output buffer.
static OutputStream nullOutputStream( )	Returns an open, but null output stream, which is a stream to which no output is written. The stream can, however, be closed. (Added by JDK 11.)
void write(int *b*)	Writes a single byte to an output stream. Note that the parameter is an **int**, which allows you to call **write( )** with expressions without having to cast them back to **byte**.
void write(byte *buffer*[ ])	Writes a complete array of bytes to an output stream.
void write(byte *buffer*[ ], int *offset*, int *numBytes*)	Writes a subrange of *numBytes* bytes from the array *buffer*, beginning at *buffer*[*offset*].

List of a few Methods defined by **outputStream** class.

Reading Console Input

Originally, the only way to perform console input was to use a byte stream, and most of Java code still uses the byte streams exclusively. Now, we can use byte or character streams. It makes our program easier to internationalize and easier to maintain.

As System.in is an instance of **InputStream**, we automatically have access to the methods defined by **InputStream**. It means we can use the **read( )** method to read bytes from **System.in**.

There are three versions of **read( )**:

int read( ) throws IOException
→This reads a single character.

int read(byte data[ ]) throws IOException
→This reads bytes from the input stream and puts them into data until either the array is full, the end of stream is reached, or an error occurs.

int read(byte data[ ], int start, int max) throws IOException
→This reads input into data beginning at the location specified by start. Up to max bytes are stored. It returns the number of bytes read, or when an attempt is made to read at the end of the stream.

Example-

```
import java.io.*;
public class ReadBytesDemo {
    public static void main(String args[]) throws IOException
```

```
    {
        byte data[] = new byte[100];
        System.out.println("Enter Some Charaters : ");
        System.in.read(data);
        System.out.print("You Entered : ");
        for(int i=0;i<data.length;i++)
        {
            System.out.print((char) data[i]);
        }
    }
}
```

Output-

Enter Some Charaters :
The quick brown fox jumps over the lazy dog.
You Entered : The quick brown fox jumps over the lazy dog.

Writing Console Output

Java originally provided only byte streams for console output. As **System.out** is a byte stream, however, byte-based console output is still widely used. Console output is most easily accomplished with **print( )** and **println( )**, with which we are already using in our programs. These methods are defined by the class.

Since **PrintStream** is an output stream derived from **OutputStream**, it also implements the low-level method **write( )** and it is possible to write to the console by using **write( )**. **PrintStream** supplies two additional output methods**: printf( )** and **format( )**. Both give us detailed control over the precise format of data that we output.

The simplest form of **write( )** defined by PrintStream is:

void write(int byteval)
→It writes the byte specified by *byteval* to the file.

Example-

```
public class WriteDemo {
    public static void main(String args[])
    {
        int a;
        a='A';
        System.out.write(a);
        System.out.write('\n');
    }
}
```


MITRC

Output-
A

We will not often use **write( )** to perform console output (although it might be useful in some situations), that's why print( ) and println( ) are substantially easier to use.

Reading and Writing Files

Java provides a number of classes and methods that allows us to read and write files. To do this, reading and writing files using byte streams is very common.

To create a byte stream linked to a file, we use **FileInputStream** or **FileOutputStream**.

Reading a File

A file is opened for input by creating a **FileInputStream** object.

FileInputStream(String fileName) throws FileNotFoundException

Here, *fileName* specifies the name of the file we want to open. If the file does not exist, then **FileNotFoundException** is thrown. **FileNotFoundException** is a subclass of **IOException**.

To read from a file, we can use **read( )** as:

int read( ) throws IOException

After reading from a file, we must close it by calling **close( )** as:

void close( ) throws IOException
→ Closing a file releases the system resources allocated to the file, allowing them to be used by another file.

Example-
Java File - ReadFileDemo.java

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class ReadFileDemo {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin;
        if(args.length != 1)
```


MITRC

```
        {
            System.out.println("Usage: ReadFileDemo File");
            return;
        }

        try
        {
            fin = new FileInputStream(args[0]);
            //fin = new FileInputStream("MyFile.txt");
        }
        catch(FileNotFoundException exc)
        {
            System.out.println("File not found");
            return;
        }

        try
        {
            //read bytes until EOF is encountered
            do
            {
                i = fin.read(); //read from the file.
                if(i != -1)
                {
                    System.out.print((char) i);
                }
            }while(i != -1);
        }
        catch(IOException exc)
        {
            System.out.println("Error reading file.");
        }

        try
        {
            fin.close(); //closing the file.
        }
        catch(IOException exc)
        {
            System.out.println("Error closing file.");
        }
    }
}
```

Text File- MyFile.txt (The file which we are reading with the java program)
The quick brown fox jumps over the lazy dog.

Output-

The quick brown fox jumps over the lazy dog.

Writing to a File

To open a file for output we need to create a **FileOutputStream** object as:

FileOutputStream(String fileName) throws FileNotFoundException
→With this form of FileOutputStream, when an output file is opened, any preexisting file by the same name is destroyed.

FileOutputStream(String fileName, boolean append) throws FileNotFoundException
→With this form of FileOutputStream, if append is **true**, then output is appended to the end of the file. Otherwise, the file is overwritten.

If the file cannot be created, then **FileNotFoundException** is thrown.

To write to a file, we will use the **write( )** method as:

void write(int byteval) throws IOException
→ This method writes the byte specified by byteval to the file. Although byteval is declared as an integer, only the loworder 8 bits are written to the file. If an error occurs during writing, an **IOException** is thrown.

Once we are done with an output file, we must close it using **close( )** as:

void close( ) throws IOException

Example-
Java File-WritetoFileDemo.java

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class WritetoFileDemo {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin = null;
        FileOutputStream fout = null;

        if(args.length != 2)
        {
            System.out.println("Usage: WritetoFileDemo from to");
            return;
        }
```


MITRC

```
        //Copy a file
        try
        {
            //Attempt to open the files.
            fin = new FileInputStream(args[0]);
            fout = new FileOutputStream(args[1]);

            do
            {
                i = fin.read(); //Read from a file
                if(i != -1 )
                {
                    fout.write(i); //Write to Other File.
                }
            }while(i != -1);
        }
        catch(IOException exc)
        {
            System.out.println("I/O Error: " + exc.getMessage());
        }
        finally
        {
            try
            {
                if(fin != null)
                {
                    fin.close();
                }
            }
            catch(IOException exc)
            {
                System.out.println("Error Closing Output File");
            }
        }
    }
}
```

Input File-MyFile.txt (which is already exists and having some data).
Output File-NewFile.txt (Which would be created by executing the java program).

LECTURE - 2

Applets: Applet Fundamentals, Applet Architecture

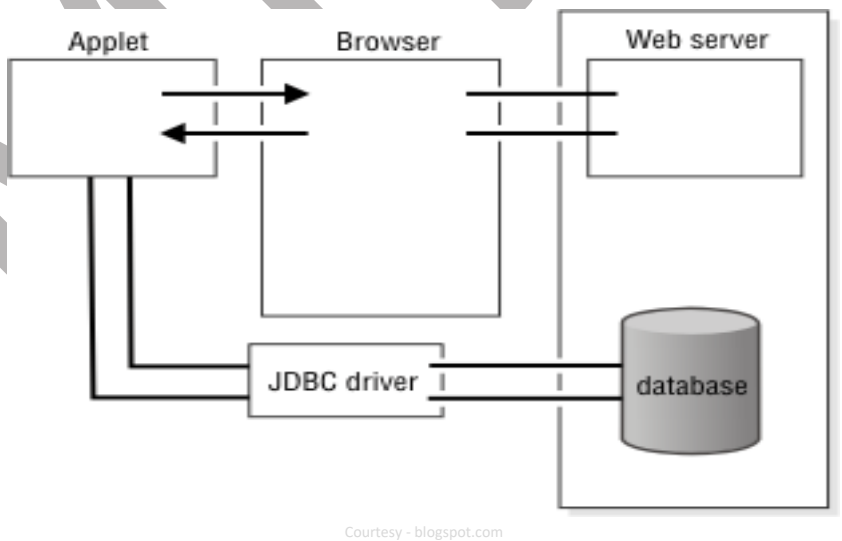
At the time of Java's creation, one of its most exciting features was the **applet**. An **applet** is a special kind of Java program that is designed to be transmitted over the Internet and automatically executed inside a Java-compatible web browser. These are the programs that you can embed in web pages to provide some intelligence like play sounds, play animations.

Important points about Applets

- Java enables us to write small programs called applets.
- An applet is a window based programming environment.
- Applete programs are java classes that extend that java.applet.Applet class and are enabaled by reference with HTML page.
- Applets are not stand-alone programs.
- Applets can't read or write local files, check for the existence of files, execute programs on the local machine, etc. These restrictions are for security reasons.

Applet Architecture

The database server is connected to the JDBC driver, which is connected to the applet. The applet is also connected to a browser, which is connected to a web server that communicates with the database.



→Because of waning browser support for applets (and other factors), JDK 9 deprecated the entire applet API and With the release of JDK 11, applet support has been removed.

LECTURE - 3

The HTML Applet Tag, Passing Parameters to Applets

Applets are embedded in HTML documents with the <APPLET> tag. It contains attributes that identify the applet to be displayed and, optionally, give the web browser hints about how it should be displayed. Attributes are name/value pairs that are interpreted by a web browser or appletviewer. Applets have both an opening <APPLET> and a closing </APPLET> tag. Sandwiched between these can be any number of <PARAM> tags that contain data to be passed to the applet:

```
<APPLET attribute
        attribute ... >
   <PARAM parameter >
   <PARAM parameter >
   ...
</APPLET>
```

The definition of the <APPLET> tag lists a fixed set of recognized attributes; specifying an incorrect or nonexistent attribute should be considered an HTML error. Three attributes are required in the <APPLET> tag. These are width, height and code or object.

Parameters

Parameters are analogous to command-line arguments; they provide a way to pass information to an applet. Each <PARAM> tag contains a name and a value that are passed as strings to the applet:

```
<PARAM name = parameter_name value = parameter_value>
```

There is no fixed set of parameter names or values. it's up to the applet to interpret the parameter name/value pairs that are passed to it. Any number of parameters can be specified, and the applet may choose to use or ignore them as it sees fit. The applet might also consider parameters to be either optional or required, and act accordingly.

How to Run Applet

There are two standard ways in which you can run an applet :

1. Executing the applet within a Java-compatible web browser.
2. Use an applet viewer to test your applet -> type at command line-
   appletviewer apletdemo.html

MITRC

Example-

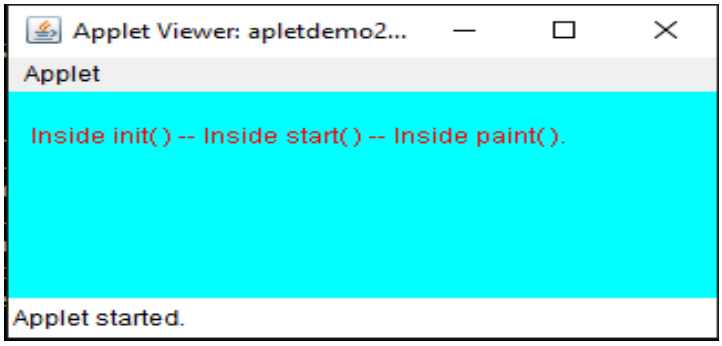
Apletdemo2.java

```
import java.awt.*;
import java.applet.*;
/*
<applet code="apletdemo2" width=300 height=50>
</applet>
*/
public class apletdemo2 extends Applet
{
    String msg;
    // set the foreground and background colors.
    public void init()
    {
        setBackground(Color.cyan);
        setForeground(Color.red);
        msg = "Inside init( ) --";
    }
    // Initialize the string to be displayed.
    public void start()
    {
        msg += " Inside start( ) --";
    }
    // Display msg in applet window.
    public void paint(Graphics g)
    {
        msg += " Inside paint( ).";
        g.drawString(msg, 10, 30);
    }
}
```

apletdemo.html

```
<html>
   <title>The Hello, World Applet</title>
   <hr>
   <applet code = "apletdemo2.class" width = "320" height = "120">
      If your browser was Java-enabled, a "Applet Demo"
      message would appear here.
   </applet>
   <hr>
</html>
```

Output-



MITRC

LECTURE - 4

Networking Basics, Java and the Net

One of the most important reasons that Java became the premier language for network programming are the classes defined in the **java.net** package.

Networking Basics

- At the core of **Java's networking** support is the concept of a **socket**. A **socket** identifies an endpoint in a network. **Sockets** are at the foundation of modern networking because a socket allows a single computer to serve many different clients at once, as well as to serve many different types of information. This is accomplished through the use of a port, which is a numbered socket on a particular machine.
- A **server** process is said to "listen" to a port until a client connects to it. A **server** is allowed to accept multiple clients connected to the same port number, although each session is unique. To manage multiple client connections, a server process must be multithreaded or have some other means of multiplexing the simultaneous I/O.
- **Socket** communication takes place via a protocol. Internet Protocol (IP) is a low-level routing protocol that breaks data into small packets and sends them to an address across a network, which does not guarantee to deliver said packets to the destination.
- **Transmission Control Protocol (TCP)** is a higher-level protocol that manages to robustly string together these packets, sorting and retransmitting them as necessary to reliably transmit data. A third protocol, **User Datagram Protocol (UDP)**, sits next to TCP and can be used directly to support fast, connectionless, unreliable transport of packets.
- **Hypertext Transfer Protocol (HTTP)** is the protocol that web browsers and servers use to transfer hypertext pages and images. It is a quite simple protocol for a basic page-browsing web server. When a client requests a file from an HTTP server, an action known as a hit, it simply sends the name of the file in a special format to a predefined port and reads back the contents of the file.
- A key component of the Internet is the **address** (IP Address). Every computer on the Internet has one address. An **Internet address** is a number that uniquely identifies each computer on the Net. Originally, all Internet addresses consisted of 32-bit values, organized as four 8-bit values which was specified by IPv4. IPv6 uses a 128-bit value to represent an address.
- As the numbers of an IP address describe a network hierarchy, the name of an Internet address, called its **domain name**, describes a machine's location in a name space.
- An Internet domain name is mapped to an IP address by the **Domain Naming Service** (DNS).

Java and the Net

The **java.net** package contains Java's original networking features. It supports TCP/IP both by extending the already established stream I/O interface and by adding the features required to build I/O objects across the network. The classes contained in the java.net package are:

Authenticator	InetAddress	SocketAddress
CacheRequest	InetSocketAddress	SocketImpl
CacheResponse	InterfaceAddress	SocketPermission
ContentHandler	JarURLConnection	StandardSocketOption
CookieHandler	MulticastSocket	URI
CookieManager	NetPermission	URL
DatagramPacket	NetworkInterface	URLClassLoader
DatagramSocket	PasswordAuthentication	URLConnection
DatagramSocketImpl	Proxy	URLDecoder
HttpCookie	ProxySelector	URLEncoder
HttpURLConnection	ResponseCache	URLPermission
IDN	SecureCacheResponse	URLStreamHandler
Inet4Address	ServerSocket	
Inet6Address	Socket	

The java.net package's interfaces are:

ContentHandlerFactory	FileNameMap	SocketOptions
CookiePolicy	ProtocolFamily	URLStreamHandlerFactory
CookieStore	SocketImplFactory	
DatagramSocketImplFactory	SocketOption	

InetAddress

The InetAddress class is used to encapsulate both the numerical IP address and the domain name for that address. We interact with this class by using the name of an IP host, which is more convenient and understandable than its IP address.

Factory Methods

The InetAddress class has no visible constructors. To create an InetAddress object, we have to use one of the available **factory methods**. Factory methods are merely a convention whereby static methods in a class return an instance of that class.

LECTURE - 5

TCP/IP Client Sockets URL, URL Connection, TCP/IP Server Sockets

TCP/IP Client Sockets

TCP/IP sockets are used to implement reliable, bidirectional, persistent, point-to-point, stream-based connections between hosts on the Internet. A socket can be used to connect Java's I/O system to other programs that may reside either on the local machine or on any other machine on the Internet, subject to security constraints.

TCP/IP Server Sockets

The **ServerSocket** class is used to create servers that listen for either local or remote client programs to connect to them on published ports. ServerSockets are quite different from normal Sockets. When we create a **ServerSocket**, it will register itself with the system as having an interest in client connections. The constructors might throw an IOException under adverse conditions.

Types of TCP Sockets

There are two kinds of **TCP sockets** in Java. One is for **servers**, and the other is for **clients**. The **ServerSocket** class is designed to be a "listener", which waits for clients to connect before doing anything. The **Socket** class is for **clients**. It is designed to connect to server sockets and initiate protocol exchanges. The creation of a **Socket** object implicitly establishes a connection between the client and server. We can gain access to the input and output streams associated with a Socket by use of the **getInputStream( )** and **getOuptutStream( )**. Each can throw an IOException if the socket has been invalidated by a loss of connection.

*Several other **methods** are available including* –

- **connect( )**-allows us to specify a new connection .
- **isConnected( )**-returns true if the socket is connected to a server.
- **isBound( )**-returns true if the socket is bound to an address.
- **isClosed( )**-returns true if the socket is closed.
- **close( )**-to close a socket.

Example –

The following example showing how to use Socket and ServerSocket Class to make connection between client and server.

ServerExp.java(Using ServerSocket)

```
import java.io.*;
import java.net.*;

public class ServerExp
{
```


MITRC

```
    public static void main(String[] args)
    {
        try
        {
            ServerSocket ss=new ServerSocket(3333);
            Socket s=ss.accept();//establishes connection

            DataInputStream dis=new DataInputStream(s.getInputStream());

            String  str=(String)dis.readUTF();
            System.out.println("message= "+str);

            ss.close();

        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

ClientExp(Using Socket)

```
import java.io.*;
import java.net.*;

public class ClientExp
{
    public static void main(String[] args)
    {
        try
        {
            Socket s=new Socket("localhost",3333);

            DataOutputStream dout=new DataOutputStream(s.getOutputStream());

            dout.writeUTF("Hello Server");
            dout.flush();

            dout.close();
            s.close();

        }catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output(On Server Machine)

```
message= Hello Server
```

URL

The **URL** provides a reasonably intelligible form to uniquely identify or address information on the Internet. URLs are ubiquitous. Every browser uses them to identify information on the Web. Within Java's network class library, the URL class provides a simple, concise API to access information across the Internet using URLs. All URLs share the same basic format, although some variation is allowed.

A **URL specification** is based on *four components*-

- The first is the protocol to use, separated from the rest of the locator by a colon (:). Common protocols are HTTP, FTP, gopher, and file, although these days almost everything is being done via **HTTP(Hypertext Transfer Protocol)** (in fact, most browsers will proceed correctly if you leave off the "http://" from our URL specification) or **HTTPS(Hypertext Transfer Protocol Secure)**.
- The second component is the host name or IP address of the host to use. This is delimited on the left by double slashes (//) and on the right by a slash (/) or optionally a colon (:).
- The third component, the port number, is an optional parameter, delimited on the left from the host name by a colon (:) and on the right by a slash (/). (It defaults to port 80, the predefined HTTP port; thus, ":80" is redundant.)
- The fourth part is the actual file path. Most HTTP servers will append a file named index.html or index.htm to URLs that refer directly to a directory resource.

Example-

```
import java.net.MalformedURLException;
import java.net.URL;

public class URLDemo {
    public static void main(String args[]) throws MalformedURLException
    {
        URL hp = new URL("https://www.google.com/WhatsNew");
        System.out.println("Protocol: " + hp.getProtocol());
        System.out.println("Port: " + hp.getPort());
        System.out.println("Host: " + hp.getHost());
        System.out.println("File: " + hp.getFile());
        //System.out.println("Ext: " + hp.getExternalForm());
    }
}
```

Output-
Protocol: https
Port: -1 → port is –1. This means that a port was not explicitly set.
Host: www.sanadvice.com
File: /WhatsNew

URLConnection

URLConnection is a general-purpose class for accessing the attributes of a remote resource. Once we make a connection to a remote server, we can use URLConnection to inspect the properties of the remote object before actually transporting it locally. These attributes are exposed by the HTTP protocol specification and, as such, only make sense for URL objects that are using the HTTP protocol. URLConnection defines several methods.

Example-

```
import java.io.InputStream;
import java.net.URL;
import java.net.URLConnection;
import java.util.Date;

public class UrlConnectionDemo {
    public static void main(String args[]) throws Exception
    {
        int c;
        URL hp = new URL("https://www.google.com");
        URLConnection hpCon = hp.openConnection();

        long d = hpCon.getDate();
        if(d==0)
        {
            System.out.println("No date information.");
        }
        else
        {
            System.out.println("Date: " + new Date(d));
        }

        System.out.println("Content-Type: " + hpCon.getContentType());

        d = hpCon.getExpiration();

        if(d==0)
        {
            System.out.println("No expiration information.");
        }
```

```
        else
        {
            System.out.println("Expires: " + new Date(d));
        }

        d = hpCon.getLastModified();
        if(d==0)
        {
            System.out.println("No last-modified information.");
        }
        else
        {
            System.out.println("Last-Modified: " + new Date(d));
        }

        long len = hpCon.getContentLengthLong();

        if(len == -1)
        {
            System.out.println("Content length unavailable.");
        }
        else
        {
            System.out.println("Content-Length: " + len);
        }

        if(len != 0)
        {
            System.out.println("=== Content ===");
            InputStream input = hpCon.getInputStream();
            while(((c = input.read()) != -1))
            {
                System.out.print((char) c);
            }
            input.close();
        }
        else
        {
            System.out.println("No content available.");
        }
    }
}
```

This program establishes an HTTP connection to www.google.com over port 80. It then displays several header values and retrieves the content.

HttpURLConnection

Java provides a subclass of **URLConnection** that provides support for HTTP connections. This class is called **HttpURLConnection**. We obtain an **HttpURLConnection** in the same way just shown, by calling **openConnection()** on a **URL** object, but we must cast the result to **HttpURLConnection**. Once we have obtained a reference to an **HttpURLConnection** object, we can use any of the methods inherited from **URLConnection**. We can also use any of the several methods defined by **HttpURLConnection**.

The URI Class

The **URI** class encapsulates a Uniform Resource Identifier (URI). URIs are similar to URLs. In fact, URLs constitute a subset of URIs. A URI represents a standard way to identify a resource. A URL also describes how to access the resource.

Cookies

The **java.net** package includes classes and interfaces that help manage **cookies** and can be used to create a stateful (as opposed to stateless) **HTTP session**. The classes are **CookieHandler**, **CookieManager**, and **HttpCookie**. The interfaces are **CookiePolicy** and **CookieStore**.

MITRC

LECTURE - 6

Database Connectivity

A database is an organized collection of data. There are many different strategies for organizing data to facilitate easy access and manipulation. A database management system (DBMS) provides mechanisms for storing, organizing, retrieving and modifying data for many users. **Database Management Systems** allow for the access and storage of data without concern for the internal representation of data. Java programs communicate with databases and manipulate their data using the **Java Database Connectivity** (**JDBC**) API.

Java Database Connectivity (JDBC)

Java Database Connectivity (JDBC) is an application programming interface for the Java programming language, which defines how a client may access a database. It is a Java-based data access technology used for Java database connectivity.

JDBC is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases. The JDBC library includes APIs for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating SQL or MySQL statements
- Executing SQL or MySQL queries in the database
- Viewing & Modifying the resulting records

JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database. Java can be used to write different types of executables, like - Java Applications, Java Applets, Java Servlets, Java ServerPages (JSPs), Enterprise JavaBeans (EJBs). JDBC provides the same capabilities as ODBC, allowing Java programs to contain database-independent code.

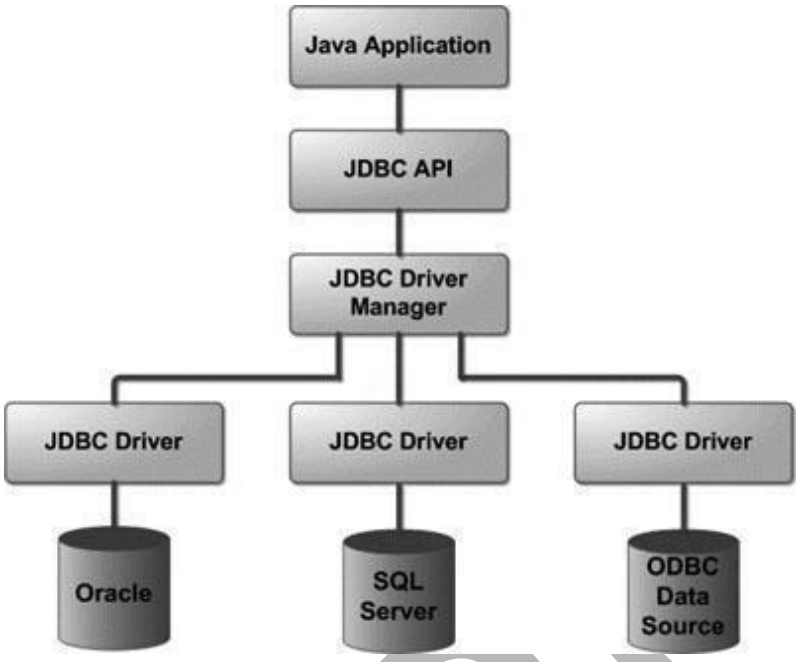
JDBC Architecture

The JDBC API supports both two-tier and three-tier processing models for database access but in general JDBC Architecture consists of two layers:

- JDBC API: This provides the application-to-JDBC Manager connection.
- JDBC Driver API: This supports the JDBC Manager-to-Driver Connection.

The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.

The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.



JDBC Architectural Diagram

Common JDBC Components

The **JDBC API** provides the following interfaces and classes:

DriverManager : This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver by using communication subprotocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.

Driver : This interface handles the communications with the database server. We will interact directly with Driver objects very rarely. Instead, we use **DriverManager** objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.

Connection : The connection object represents communication context, i.e., all communication with database is through connection object only.

Statement : We use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.

ResultSet : These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.

SQLException : This class handles any errors that occur in a database application.

MITRC

Relational Databases

Most popular database systems are **relational databases**. A relational database is a logical representation of data that allows the data to be accessed without consideration of its physical structure. A relational database stores data in **tables**. Tables are composed of **rows**, and rows are composed of **columns** in which values are stored. One column of each row can be the table's **primary key** – a column (or group of columns) with a **unique value** that cannot be duplicated in other rows. This guarantees that each row can be identified by its **primary key**.

Structured Query Language (SQL)

A language called **Structured Query Language** (**SQL)** - pronounced "sequel," or as its individual letters - is the international standard language used almost universally with relational databases to perform queries (i.e., to request information that satisfies given criteria) and to manipulate data.

SQL Keywords	Description
SELECT	Retrieves data from one or more tables.
FROM	Tables involved in the query. Required in every SELECT.
WHERE	Criteria for selection that determine the rows to be retrieved, deleted or updated. Optional in a SQL query or a SQL statement.
GROUP BY	Criteria for grouping rows. Optional in a SELECT query.
ORDER BY	Criteria for ordering rows. Optional in a SELECT query.
INNER JOIN	Merge rows from multiple tables.
INSERT	Insert rows into a specified table.
UPDATE	Update rows in a specified table.
DELETE	Delete rows from a specified table.

Basic SELECT Query - A SQL query "selects" rows and columns from one or more tables in a database. The basic form of a SELECT query is-

SELECT * FROM tableName

WHERE Clause - It's necessary to locate rows in a database that satisfy certain selection criteria. Only rows that satisfy the selection criteria (formally called predicates) are selected. SQL uses the optional WHERE clause in a query to specify the selection criteria for the query. The basic form of a query with selection criteria is-

SELECT columnName1, columnName2, … FROM tableName WHERE criteria

ORDER BY Clause - The rows in the result of a query can be sorted into ascending or descending order by using the optional ORDER BY clause. The basic formof a query with an ORDER BY clause is-

SELECT columnName1, columnName2, … FROM tableName ORDER BY column ASC
SELECT columnName1, columnName2,…FROM tableName ORDER BY column DESC

Merging Data from Multiple Tables: INNER JOIN - It's necessary to merge data from multiple tables into a single result. Referred to as joining the tables, this is specified by an INNER JOIN operator, which merges rows from two tables by matching values in columns that are common to the tables. The basic form of an INNER JOIN is-

SELECT columnName1, columnName2, …
FROM table1
INNER JOIN table2
ON table1.columnName = table2.columnName

INSERT Statement - The INSERT statement inserts a row into a table. The basic form of this statement is-

INSERT INTO tableName (columnName1, columnName2, …, columnNameN)
VALUES (value1, value2, …, valueN)

UPDATE Statement - An UPDATE statement modifies data in a table. Its basic form is-
UPDATE tableName
SET columnName1 = value1, columnName2 = value2, …, columnNameN = valueN
WHERE criteria

Creating JDBC Application

There are following seven steps involved in building a JDBC application:

- **Import the packages** - Requires that we include the packages containing the JDBC classes needed for database programming. Most often, using import java.sql.* will suffice.

- **Register the JDBC driver** . Requires that we initialize a driver so we can open a communications channel with the database.

- **Open a connection** . Requires using the DriverManager.getConnection() method to create a Connection object, which represents a physical connection with the database.

MITRC

- **Preparing a query** . Create an object of prepareStatement to assign a SQL statement to execute a query.

- **Execute a query** . Requires using an object of type Statement for building and submitting an SQL statement to the database.

- **Extract data from result set** . Requires that we use the appropriate ResultSet.getXXX() method to retrieve the data from the result set.

- **Clean up the environment** . Requires explicitly closing all database resources versus relying on the JVM's garbage collection.

Example-

```
//Setp 1 - Import required packages
import java.sql.*;

class mysqlcon
{
    public static void main(String args[])
    {
        try
        {
            //Step 2 - Register JDBC driver
            Class.forName("com.mysql.jdbc.Driver");

            //Step 3 - Open a connection
            Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/stdmarks", "root",
"root");

            //Step 4 - Preparing Query
            PreparedStatement ps = con.prepareStatement("select * from stddata
where perc > 60");

            //Step 5 - Execute a Query
            ResultSet rs = ps.executeQuery(); //execute

            System.out.println("Students having marks > 60 are:");

            //Step 6 - Extracting Data
            while(rs.next())
                System.out.println(rs.getString(2));

             //Step 7 - Clean-up environment
            con.close();
        }
```

```
        catch(Exception e)
        {
            //Exception Handling
            System.out.println("Error" + e.toString());
        }
    }
}
```

The above example extracting data from a mysql database stdmarks and executing a query which retrive data of those students having marks greater than 60.

Example(Inserting Data in Database using JDBC)-

```
import java.sql.*;
class mysqlinsdata
{
    public static void main(String args[])
    {
        try
        {

            Class.forName("com.mysql.jdbc.Driver");
            Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/stdmarks", "root",
"root");
            //Statement stat = con.createStatement();
            String sql = "insert into stddata (stdname, stdage, stdm1, stdm2,
stdm3, totmarks, perc, results) values('bcd',18,74,66,80,216,73.33,'First')";
            PreparedStatement pst =  con.prepareStatement(sql);
            int numRowsChanged = pst.executeUpdate(sql);
            System.out.println(numRowsChanged);
            //ResultSet rs = stat.executeUpdate("insert into stdmarks (stdname,
stdage, stdm1, stdm2, stdm3, totmarks, perc, results)
values('abc',18,70,66,80,216,72.0,'First')");
            con.close();
        }
        catch(Exception e)
        {
            System.out.println("Error" + e.toString());
        }
    }
}
```

In this example we are inserting data in an existing database.